CLOUD BIGTABLE SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

Question: 1. You need to store IoT sensor readings with device IDs and timestamps while supporting queries for the last 24 hours per device. How would you design the row key?

Answer

Step 1. Describe the goals you must satisfy - low-latency writes from many devices, efficient per-device scans for “last 24 hours”, and avoidance of Bigtable hotspotting caused by strictly sequential keys; keeping goals clear guides the key design.

Step 2. Use a stable, fixed-width device identifier as the prefix - choose a compact deviceID (for example zero-padded numeric ID or a compact UUID without hyphens) so rows for the same device colocate and range scans by deviceID are straightforward.

Step 3. Add a time component that orders recent data first using a reversed timestamp - compute reversed_ts = MAX_TIMESTAMP - unix_milliseconds and encode it as a fixed-width decimal or fixed-size big-endian bytes so lexicographic order corresponds to newest→oldest.

Step 4. Compose the row key as deviceID + delimiter + reversed_timestamp - for example: device123|18446744073709551615 - 169xyz (with the actual reversed numeric value substituted); using a clear delimiter and fixed-width encoding prevents accidental collisions.

Step 5. Show how to query the last 24 hours - compute unix_ms_24h_ago, convert to reversed_lower = MAX_TIMESTAMP - unix_ms_24h_ago, then scan the row range for deviceID from start = deviceID|000... to end = deviceID|reversed_lower; because keys are newest-first you can stop scanning when rows are older than 24 hours.

Step 6. Add practical operational notes - set column-family TTL/GC to expire older data, choose consistent encoding and delimiters, monitor per-device throughput and consider sharding a single very-high-volume device if it still hotspots (see Question 2), and test with representative load to confirm range-scan performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 2. Your Bigtable workload is suffering from hotspotting due to sequential row keys. How would you fix it?

Answer

Step 1. Diagnose the hotspot cause - verify that writes concentrate on a narrow key range (e.g., sequential timestamps or monotonically increasing prefixes) by checking per-node CPU, latency, and tablet-server metrics; hotspotting results when a single tablet receives most of the writes.

Step 2. Remove strict monotonicity by reversing or moving the timestamp - use reversed timestamps (MAX - ts) so new writes don't always append to the same increasing byte sequence, or place the time after a high-cardinality prefix to distribute load.

Step 3. Introduce deterministic sharding (salting) for high-write keys - prepend a small shard id (for example hash(deviceID) % N) so keys become <shard>#<deviceID>|<time>. This spreads writes across N tablets while remaining predictable for reads.

Step 4. Explain the read tradeoff - when you shard, reads for a single device's last 24 hours must query all N shards in parallel and merge results client-side - this increases read complexity but prevents write-side overload; choose N to balance write distribution and read cost.

Step 5. Consider time-bucketing as an alternative or complement - include a time-bucket component like yyyyMMddHH in the key (deviceID|yyyyMMddHH|reversed_ts) so bursts are spread across hourly buckets and reads only need the recent few buckets for a 24-hour window.

Step 6. Operational recommendations and tuning - start with modest shard count (8 or 16), use deterministic hashing, pre-split the table if you can predict hot key ranges, enable autoscaling and monitor tablet splits, batch writes, and apply GC/TTL to cap table size.

Step 7. Combine approaches when needed - a good production pattern is reversed-time + moderate sharding or time-bucketing for very hot devices, plus monitoring and iterative tuning so you minimize hotspots while keeping reads efficient.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 3. You need to decide between using wide tables (many columns) vs tall tables (many rows) for a recommendation engine. How would you choose?

Step 1. Start from access patterns - determine exactly how the application will read and write data (for example: "fetch latest 100 recommendations for a user", "update a single item score", "compute offline model over all interactions") because Bigtable schema should be driven by queries; choose the layout that makes the most common queries single-row or single-range operations.

Step 2. Evaluate cardinality and sparsity - if the item space is very large and each user interacts with only a tiny fraction (highly sparse), a tall schema (one interaction per row) usually wins because you avoid huge sparse rows and keep row sizes small. If each user needs fast random reads of many specific attributes for a moderate, bounded set of items, a narrow-wide per-user row can be OK.

Step 3. Consider update and atomicity needs - Bigtable guarantees atomicity within a single row. If your workflow needs to update many item fields together atomically (for a single user), a wide row (all those columns in the same row) makes atomic updates simple. If updates are independent per interaction or item, tall (one interaction row) simplifies concurrent writes.

Step 4. Respect row-size and performance limits - keep rows reasonably small (aim well under Bigtable's practical limits; avoid single rows growing to many megabytes). Wide tables risk very large rows if you store long item lists per user; choose tall or time-windowed wide rows (see Step 6) if row size could blow up.

Step 5. Optimize for read cost and simplicity - wide rows let you read one row to get many columns (cheap if you need everything for that user). Tall rows require range scans or secondary lookups (more RPCs). If low-latency single-RPC reads per user are critical and the per-user column count is bounded, wide is preferable.

Step 6. Use hybrid approaches for practicality - combine both patterns: maintain a tall, append-only table of interactions for analytics and model training, and maintain a denormalized wide table (or a cached per-user row) with top-N recommendations, recent interactions, or precomputed features for fast serving. Use TTL or time-buckets to keep the wide serving rows bounded in size.

Step 7. Plan for indexing and joins - Bigtable is not relational; if you need reverse lookups (item→users) or complex joins, create additional tables keyed for those lookups (tall tables keyed by item or user-item). Avoid trying to force multi-index queries into one schema — denormalize into targeted tables.

Step 8. Operational guidelines and monitoring - pre-split the table based on expected key distribution, design row keys to avoid hotspots, set column-family GC/TTL for retention, and monitor latency/throughput. Start with the schema that optimizes your hottest production path (serve latency or bulk analytics), and adjust using the hybrid pattern if you need both high write throughput and low-latency reads.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 4. You must bulk load 10 years of historical data from GCS into Bigtable. How would you design this ingestion pipeline?

Step 1. Prepare and validate the source data in GCS - normalize formats (Parquet/Avro or newline-delimited JSON/CSV), validate schema, clean corrupt rows, canonicalize device/user IDs and timestamps, and produce a sample of keys to analyze key distribution before loading.

Step 2. Choose and implement the row-key transform early - compute the final Bigtable row key in the pipeline (applying your production row-key design: deviceID/time-bucket/reversed_ts or sharded key). Doing this transformation first lets you sample keys to create split keys and avoid hotspots during the load.

Step 3. Partition the input for parallelism - split the GCS data into many independent shards (files or file ranges) so you can run many parallel workers. Partitioning strategy should align with your row-key distribution (for example, partition by hashed key prefix or by time ranges if you use time-bucketing).

Step 4. Use Dataflow (Apache Beam) or a managed bulk-write tool - implement a distributed pipeline that reads from GCS, applies transforms, batches mutations, and writes to Bigtable using the Cloud BigtableIO / BulkMutation facilities. Configure the pipeline to use large parallelism, but respect Bigtable write best-practices (batch sizes, per-connection concurrency).

Step 5. Pre-split the Bigtable and set appropriate instance resources - create the destination table ahead of time and pre-create split keys derived from the sampled key distribution so tablets are balanced. Provision enough nodes (or configure autoscaling) to handle the expected ingest throughput without creating hotspots.

Step 6. Tune batching, throttling, and retries - batch mutations into reasonably large BulkMutations to reduce RPC overhead, but avoid oversized requests that trigger timeouts. Implement exponential backoff on transient errors, and use client-side rate control to avoid overwhelming Bigtable or incurring hotspotting on popular key ranges.

Step 7. Load in controlled windows and verify incrementally - ingest in parallel but in checkpoints (for example per-day or per-month chunks) and verify each chunk with row-count checksums, sampling, or end-to-end test reads. This lets you detect issues early and rollback or reprocess limited ranges instead of restarting the entire 10-year load.

Step 8. Monitor and optimize during the run - watch Bigtable CPU, latency, and per-tablet metrics; observe Dataflow worker logs; and be ready to adjust parallelism, add nodes, or change batch sizes. If you see hotspots, pause and adjust the key transform (increase shards or change partitioning) and replay affected partitions.

Step 9. Post-load housekeeping - set column-family GC rules/TTL to enforce retention, run integrity checks (compare row counts, checksums, spot-check values), create or update secondary tables or indexes needed for serving, and remove any temporary staging artifacts in GCS.

Step 10. Plan for repeatability and rollback - keep the pipeline code and run parameters in version control, store run manifests (which files were ingested), and design idempotent ingestion (use dedupe keys or use a watermark) so you can safely re-run or roll back parts of the load without corrupting production data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 5. Real-time financial transactions must flow from Pub/Sub into Bigtable with sub-second latency. How would you implement this?

Step 1. Clarify latency and consistency goals and design constraints - decide what "sub-second" means (e.g., 500 ms end-to-end), whether exactly-once or at-least-once delivery is required, throughput (tx/sec), and retention/TTL; these requirements determine trade-offs for batching, retries, and deduplication.

Step 2. Use Pub/Sub as the ingest front door with small publish and ack latency - producers publish transactions to a topic; keep message size small, use synchronous or low-latency async publishes, and enable batching only with very small batch windows to avoid added latency.

Step 3. Build a low-latency streaming consumer (Dataflow/Beam or autoscaled fleet) - run a streaming pipeline that subscribes to Pub/Sub with high parallelism and low ack-deadline, decodes and validates messages, computes Bigtable row keys (applying your row-key design for fast lookups and to avoid hotspots), and performs small, frequent write batches to Bigtable.

Step 4. Optimize Bigtable writes for latency - use the Bigtable client's asynchronous bulk mutation API with tuned batch sizes (small batches to minimize queuing yet large enough to amortize RPCs), keep per-connection concurrency high, and pin hot key ranges across sufficient nodes or use sharding/time-buckets to avoid write hotspots that would inflate latency.

Step 5. Implement deduplication / idempotency for financial correctness - include a stable transaction ID in each message and either write using conditional checks / check-and-set semantics where appropriate, or store a processed-tx column and have the consumer check-and-write atomically within the same row to avoid double-application on retries.

Step 6. Tune flow-control, retry/backoff, and SLA-aware throttling - implement client-side rate limiting and adaptive backoff for Bigtable throttling responses, keep Pub/Sub ack deadlines and streaming buffer sizes tuned to avoid message pile-up, and surface backpressure signals so upstream producers can slow down if Bigtable becomes the bottleneck.

Step 7. Monitor, test, and run chaos experiments - instrument end-to-end latency (Pub/Sub → consumer → Bigtable), per-tablet latencies, RPC latencies, CPU, and hotspots; load-test the full path to verify sub-second SLO under realistic peaks; and add alerts and autoscaling policies (Dataflow workers or consumer instances, and Bigtable nodes) to maintain the SLO.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 6. You want to support incremental ingestion where only changed records are written into Bigtable. How would you approach it?

Step 1. Choose a change-detection strategy - decide whether the source system can emit change events (CDC), or you must diff snapshots; preferred is CDC (e.g., change stream, Debezium, or Pub/Sub events) because it provides per-row deltas with timestamps and reduces unnecessary work.

Step 2. Include a stable primary key and version/timestamp in each change event - every event should carry the record key and a monotonically increasing version or last-modified timestamp so the consumer can determine whether an incoming change is newer than what's stored.

Step 3. Implement consumer-side idempotency and conditional writes - before writing to Bigtable, compare the incoming version/timestamp with the stored value (read-modify-write or use conditional mutations) and only apply the mutation if the incoming version is newer; this avoids rewriting unchanged data and handles out-of-order delivery.

Step 4. Use compact, delta-only mutations when possible - instead of replacing entire rows, send only changed columns as SetCell mutations or use increment/decrement operations for counters to minimize network and server work; smaller mutations reduce write amplification and speed.

Step 5. Handle deletes and tombstones consistently - when source signals a delete, write a tombstone column or delete the row using Bigtable mutations (and rely on GC rules/TTL to physically remove data later); ensure deletions carry versions so stale deletes don't override newer inserts.

Step 6. Optimize reads for comparison with lightweight checks - avoid full-row reads on every event by keeping a small metadata column (latest_version or last_updated_ms) in each row; a single-cell read is cheap and lets you decide whether to fetch more or apply changes, minimizing read cost.

Step 7. Support batching and windowed deduplication for performance - coalesce multiple changes for the same key in short windows in the consumer (e.g., 100–500 ms) to merge them into a single mutation; this reduces write load while preserving near-real-time behavior.

Step 8. Ensure eventual consistency and reconciliation - schedule periodic reconciliation jobs that scan the source and Bigtable (or compare snapshots) to catch missed or out-of-order updates; keep an ingestion manifest or offsets so you can replay or reprocess ranges idempotently.

Step 9. Monitor correctness and latency metrics - track the rate of applied vs. received events, dropped/duplicate events, lag between source change timestamp and Bigtable commit, and set alerts for abnormal behavior so you can react before data becomes inconsistent.

Step 10. Make the pipeline observable and re-runnable - persist processing offsets (Pub/Sub ack ids or CDC positions), log failures with contextual keys, and design the pipeline to replay ranges safely (idempotent writes and version checks) so incremental ingestion is robust and maintainable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 7. A business user needs to fetch a single customer record by ID instantly. How would you design the query and schema?

Step 1. Clarify access pattern and SLAs - confirm that the dominant operation is single-row lookup by customer ID with strict low-latency SLO (e.g., sub-10ms), and note any update patterns (frequency of writes/updates), row size limits, and whether atomic multi-field updates are required.

Step 2. Make the customer ID the row key (fixed-width, canonical form) - use a stable, compact encoding (for example zero-padded numeric ID or UUID without hyphens) so one row equals one customer and a single GetRow RPC returns the entire record quickly.

Step 3. Put hot/volatile attributes in a narrow column family and large/rarely-read blobs elsewhere - store profile fields (name, contact, status, last_login, version) in one column family for fast reads; store large lists (order history, documents) in separate tables or column families to keep the main customer row small and fast.

Step 4. Include a small metadata cell for versioning and idempotency - keep a single-cell last_updated_ms or version number in the main row so clients and incremental pipelines can cheaply check staleness with a single-cell read before writing or re-fetching.

Step 5. Provide secondary lookup tables if needed - if users sometimes search by email/phone, maintain lightweight lookup tables keyed by email→customerID and phone→customerID so the two-step read (lookup then GetRow) stays fast; keep these lookup rows tiny and write them transactionally from the application.

Step 6. Design row-key distribution to avoid hotspots - if customer IDs are sequential and writes are heavy, add a deterministic salt or use hashed IDs to spread load; if IDs are already high-cardinality (UUID), keep them as-is for direct single-row access.

Step 7. Tune Bigtable and client settings for single-row latency - provision enough Bigtable nodes (or autoscaling), use the native async Bigtable client with connection pooling, keep rows small (< few hundred KB ideally), and ensure the application uses point lookups (ReadRow) rather than scans.

Step 8. Operationalize with monitoring and fallback - instrument read latency, cache very hot customer rows in an in-memory cache (e.g., Memorystore/Redis) if needed, set GC/TTL policies for aged data, and provide an out-of-band reconciliation path for rare stale reads.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 8. Analysts need to run range queries for events between two timestamps. How would you model the row key?

Step 1. Capture query shape precisely - determine whether analysts query across all entities (global time range) or per-entity (events for a specific customer/device between timestamps), whether they expect ascending or descending order, and the expected time-window sizes (minutes/hours/days).

Step 2. Make timestamp the leading component when global time-range scans are common - encode time as fixed-width big-endian unix milliseconds (or ISO8601 like YYYYMMDDHHMMSS with zero-padding) so lexicographic order equals chronological order and a range scan between two timestamps is a contiguous key range.

Step 3. Add a stable tie-breaker after the timestamp - append eventID or entityID after the timestamp (for example: <yyyyMMddHHmmssfff>|<entityID>|<eventID>) so multiple events in the same millisecond sort deterministically and rows are unique.

Step 4. Use time-bucketing to limit scan breadth and improve locality - prefix keys with a coarse bucket (e.g., yyyyMMdd or yyyyMMddHH) and then the fine-grained timestamp: <yyyyMMdd>|<HHmmssSSS>|<entityID>. Analysts scanning a 2-day range only need to scan the buckets covering those days rather than the entire table.

Step 5. Choose ordering based on typical analysis - if analysts usually want newest-first, use reversed timestamps (MAX_TS - ts) as the leading component; otherwise use normal big-endian timestamps for chronological ascending scans. Pick the one that minimizes client-side work for the common case.

Step 6. If frequent per-entity time-range queries are needed, include entity prefix for those cases - for queries like “events for device X between T1 and T2”, use entityID|<yyyyMMddHHmmss> so per-entity scans are contiguous and cheap; combine this with a separate global time-index table for cross-entity analytics.

Step 7. Plan for sharding and pre-splitting to avoid hotspots - if many events share the exact timestamp prefix or a single bucket becomes hot, add a small deterministic salt or pre-split tablets by the sampled key distribution so writes and scans remain balanced.

Step 8. Operational considerations - maintain GC/TTL for aging event data, build helper utilities that translate analyst-friendly time ranges into the corresponding key ranges (including which buckets to scan), provide parallel scanning across buckets for performance, and monitor per-tablet metrics to adjust bucket size or sharding if needed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 9. Your team needs secondary indexes for querying by email, but Bigtable doesn't support indexes.

Step 1. Create a separate lightweight index table that maps the secondary key to the primary key - design a table (for example email_to_user) where the row key is the normalized email (lowercased, trimmed) and the value is the primary customerID (or pointer to the primary row). A single GetRow on this table returns the customerID quickly.

Step 2. Keep index updates atomic and idempotent from the application side - when creating or updating a user, perform two mutations: write the primary user row and write the index row. Use client-side retry-safe logic and idempotent operations; where you need uniqueness (email must be unique) use a check-and-mutate on the index row to ensure you don't overwrite an existing mapping.

Step 3. Use compare-and-mutate (check-and-set) patterns to enforce uniqueness - implement a conditional mutation on the index table row that only writes email→customerID if the row is empty (or has an expected value). This prevents race conditions for concurrent creates; if it fails, surface a handled conflict to the calling service.

Step 4. Handle email updates and deletes carefully with ordering/versioning - on email change: (a) write the new index row with a compare-and-mutate ensuring no collision, (b) update the primary row to reference new email/version, (c) delete the old index row. Use a version or last_updated cell in the primary row to detect and avoid stale overwrites if operations happen out-of-order.

Step 5. Provide strong operational safety via retries and compensating actions - because Bigtable lacks multi-row transactions, design your write path to be idempotent and include compensating steps: if primary write succeeds but index write fails, retry the index write or enqueue a reconciliation job (DLQ) to fix orphans; log operations with trace IDs for debugging.

Step 6. Support lookups by email in two fast steps - client flow: lookup email_to_user by email → get customerID → ReadRow on users table by customerID. Keep lookup rows small (single cell) to keep the two-RPC flow low-latency.

Step 7. Build a background reconciliation process and DLQ - run a periodic Dataflow/Batch job that scans users and index table to detect mismatches (orphaned index rows, missing index rows, or duplicate emails) and repair them, and ensure ingestion pipeline writes failures are sent to a DLQ (GCS/Pub/Sub) for later replay.

Step 8. Consider sharding and pre-splitting for hot emails - if certain emails or domains cause hotspots (e.g., many creates/reads), prefix the index row with a deterministic shard or hash when necessary and make the lookup deterministic (query known shard(s)); prefer deterministic hashing so reads know which shard to check.

Step 9. Plan monitoring, metrics, and TTLs - instrument index write failures, reconciliation mismatches, lookup latencies, and ensure GC/TTL rules are set if you want stale indexes cleaned automatically; keep alerts for high error rates so you can react quickly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 10. You need to build a pipeline where streaming events from Pub/Sub flow into Bigtable through Dataflow. How would you design this?

Step 1. Define the event contract and SLAs up front - specify message schema (JSON/Avro/Protobuf), required fields (primary key, event_ts, event_id), latency SLO, throughput, and idempotency requirements (transaction IDs or sequence numbers) so the pipeline can guarantee correctness and performance.

Step 2. Use Pub/Sub topic + subscription tuned for streaming - producers publish to a Pub/Sub topic; create a subscription with appropriate ack-deadline and enable message ordering only if your downstream uses it and you can shard by ordering key; otherwise rely on independent parallelism.

Step 3. Implement a streaming Dataflow (Apache Beam) pipeline as the consumer - the pipeline should: read from Pub/Sub, validate/parse messages, enrich/transform events (compute Bigtable row key, normalize fields, attach processing metadata), and emit Bigtable Mutations. Use the Cloud Dataflow runner in streaming mode for autoscaling and low-latency processing.

Step 4. Compute and use a production row-key strategy inside the pipeline - derive row keys that avoid hotspots (device/user prefix + reversed_ts, or sharded prefix + key), and ensure the pipeline applies the same deterministic transform so writes are balanced across Bigtable tablets.

Step 5. Batch and write mutations to Bigtable efficiently but with low latency - use BigtableIO or the Bigtable client with small-to-moderate batch windows (micro-batching) to amortize RPCs while keeping end-to-end latency sub-second; tune batch size, max outstanding RPCs, and use asynchronous buffered writes.

Step 6. Make writes idempotent and resilient to retries - include event_id or version in the mutation and perform idempotent patterns (overwrite only if incoming version >= stored version, or maintain a processed-event column). If conditional mutations are needed, use check-and-mutate on the same row to avoid double-application.

Step 7. Provide robust error handling and a dead-letter path - on transform or write failures, route messages to a DLQ (Pub/Sub topic or GCS) with failure reason and context; implement retry/backoff logic in Dataflow and limit retries before sending to DLQ to avoid poison-message loops.

Step 8. Pre-split and provision Bigtable for ingest - before heavy streaming, pre-split the table using sampled keys or create pre-defined split keys, provision enough nodes (or enable autoscaling) to handle expected write TPS, and monitor tablet hotness to adjust sharding or capacity.

Step 9. Observe and autoscale components - emit metrics (Pub/Sub ack lag, Dataflow processing latency, Bigtable mutation latency, per-tablet CPU), set alerts, and allow Dataflow autoscaling and Bigtable node scaling (manual or autoscaler) to react to load spikes. Also expose tracing for message flow (publish → Dataflow → Bigtable) for debugging.

Step 10. Test end-to-end and enable replayability - implement tests and chaos runs to validate latency and correctness; persist processing offsets or use Pub/Sub acknowledgements carefully so you can replay ranges from Pub/Sub (or replay source exports) and make the pipeline idempotent so replays do not corrupt Bigtable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 11. Your Dataflow job is writing to Bigtable but throughput is low. What optimizations would you apply?

Step 1. Measure and identify the bottleneck - collect end-to-end metrics (Dataflow shuffle/worker CPU, Pub/Sub ack lag, Bigtable mutation latency, per-tablet hotness, RPC error/429 rates). Knowing whether the bottleneck is client-side, network, Bigtable nodes, or hotspots determines which optimizations to apply.

Step 2. Increase pipeline parallelism and right-size workers - raise the number of Dataflow workers (or use autoscaling with a higher max), increase worker CPU/memory types if CPU-bound, increase number of Bigtable client threads per worker, and tune Beam parallelism (maxConcurrency, bundle size) so more mutations are issued concurrently.

Step 3. Batch mutations efficiently (micro-batching) - group many SetCell/DeleteRow mutations into moderate-sized BulkMutations to amortize RPC overhead. Tune batch size so requests are large enough to be efficient but small enough to keep end-to-end latency acceptable (start with hundreds to a few thousand cells per batch and iterate).

Step 4. Use the async Bigtable client and keep many outstanding RPCs - ensure your writes use non-blocking/asynchronous APIs with a high max outstanding RPCs setting so workers don't block waiting for individual calls; tune the client pool and channel reuse to reduce connection churn.

Step 5. Fix row-key hotspots and pre-split the table - analyze key distribution; redesign keys (salting/sharding, reversed timestamps, time-buckets) if writes concentrate on a narrow range. Pre-create split keys from a sampled distribution so tablets are balanced before the load begins.

Step 6. Provision more Bigtable capacity and enable autoscaling - increase node count or enable Bigtable autoscaling during the load so the cluster can handle higher throughput; monitor per-node CPU/IO and increase nodes if Bigtable is the limiting factor.

Step 7. Tune Dataflow-to-Bigtable interaction parameters - adjust BigtableIO parameters (maxBatchSize, maxInflightRequests, maxRowKeyLen), set sensible retry/backoff settings to avoid long stalls, and consider per-worker connection limits to avoid server-side connection throttling.

Step 8. Align partitioning between input and output - shard input files or Dataflow bundles by the same key prefix or hash used in row keys so each worker writes to many tablets rather than all workers hitting the same tablet; this reduces contention and improves throughput.

Step 9. Optimize mutation payloads and serialization - send only changed columns (delta updates), avoid large blobs in hot paths, compress payloads if network bound, and use efficient binary formats (Avro/Protobuf) during pipeline transforms to reduce CPU and network cost.

Step 10. Iterate with load tests and continuous monitoring - run controlled load tests, observe tablet-level metrics and Dataflow worker profiles, then incrementally tune batching, concurrency, key design, and Bigtable nodes until throughput meets targets while keeping latency and error rates acceptable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 12. Duplicate messages are causing inconsistent data in Bigtable. How would you ensure idempotency in Dataflow → Bigtable writes?

Step 1. Adopt an idempotency design principle up front - decide whether you will use a unique event ID, a monotonically increasing version/timestamp, conditional writes (compare-and-mutate), or a persistent dedupe index. Each has tradeoffs in storage, latency, and complexity.

Step 2. Include a stable unique event identifier and a version/timestamp in every message - require producers to emit a transaction/event_id and a last_updated_ts or monotonic version so the consumer can detect duplicates and ordering without fetching full rows.

Step 3. Use Bigtable compare-and-mutate for same-row dedupe/conditional apply - store a small metadata cell (last_event_id or last_version) in the target row and perform a CheckAndMutateRow: check that stored_version < incoming_version (or that last_event_id != incoming_id) then apply mutations. This makes acceptance atomic on a per-row basis and prevents duplicate application on retries.

Step 4. Keep dedupe metadata compact and cheap to read - avoid full-row reads to check versions by maintaining a single-cell last_updated_ms or last_event_id; a single-cell read is cheap and lets you decide whether to issue a conditional mutation, reducing unnecessary work.

Step 5. Use a persistent dedupe index for cross-row or long window deduplication - when events for many keys or long dedupe windows are needed, write an idempotency record keyed by event_id (for example, a separate processed_events table with event_id → processed_ts). Before applying a business mutation check-and-create the idempotency row (compare-and-mutate to insert only if missing) so duplicates are rejected globally.

Step 6. Make operations idempotent by design where possible - design mutations so repeated application produces the same result (e.g., SetCell with the same value, upserts keyed by event version). For counters or increments, convert to stored absolute values or use idempotent conditional increments with recorded event IDs.

Step 7. Use short in-memory caches for hot, recent dedupe and a persistent fallback - maintain a worker-local LRU cache for the last N event_ids to avoid extra RPCs for extremely high-rate duplicates; on cache miss fall back to persistent check (metadata cell or processed_events table) to ensure correctness.

Step 8. Handle out-of-order delivery with version checks - when events can arrive out of order, use monotonically increasing versions or last_updated_ts and only apply an incoming write if incoming_version >= stored_version. Log or DLQ older/out-of-order events for audit/reconciliation.

Step 9. Provide a dead-letter and reconciliation process - route messages that repeatedly fail dedupe checks or conditional mutations to a DLQ with context, and run periodic reconciliation jobs to repair missed or partial updates by scanning source vs Bigtable and reapplying idempotent operations.

Step 10. Test and monitor idempotency behavior - create tests that replay messages multiple times, verify no double-application, monitor metrics (duplicate rejection rate, check-and-mutate failures, DLQ volume), and alert on unusual patterns so you can adjust dedupe windows, metadata retention, or index sizing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 13. You must run a Spark MLlib model using data stored in Bigtable. How would you set up the integration?

Step 1. Clarify data volume, schema, and access pattern - determine whether you need to train on a full historical dataset (large, batch) or incremental/online updates, what features/columns are needed, row-key design, and whether reads are per-row or wide scans; this decides whether to read directly from Bigtable or export to GCS first.

Step 2. Choose the read path — direct connector or export - for moderate-size jobs prefer the Cloud Bigtable HBase client (HBase API) or an open-source Spark↔Bigtable connector so Spark can read rows directly; for very large training sets or heavy parallelism consider bulk-exporting Bigtable to Avro/Parquet on GCS (using Dataflow or a MapReduce job) and run Spark over files for better IO throughput.

Step 3. Provision Dataproc/Spark with Bigtable client jars and credentials - create a Dataproc cluster (or Spark cluster) with the Cloud Bigtable HBase client and connector jars on the classpath, attach a service account with Bigtable read permissions, and set Hadoop/Spark configuration (project, instance, credentials) so the connector can open scans.

Step 4. Design efficient scans and partitioning - implement row-range scans using start/stop keys or time-bucketed prefixes to split the work into many independent partitions; set the HBase scan caching (scanner.caching) and batch sizes to reduce RPC overhead and ensure each Spark partition reads a balanced key range to avoid hotspots.

Step 5. Read into Spark DataFrame and perform feature engineering - map Bigtable cells to a flat schema (use necessary columns only), convert to DataFrame/RDD, apply joins/enrichments, impute/normalize features, and persist intermediate datasets (cache/persist) in memory or spill to Parquet on GCS if memory-limited.

Step 6. Train with MLlib using tuned cluster resources - choose appropriate executors, cores, and memory (use Dataproc autoscaling if available), use MLlib algorithms that scale (e.g., MLlib's distributed algorithms), and pipeline steps that avoid serial bottlenecks; checkpoint and persist model artifacts to GCS.

Step 7. Write model outputs / predictions back safely - for per-row predictions or model metadata, write results back to Bigtable using the same connector and batched buffered mutations (BufferedMutator) with retry/backoff; ensure idempotency (event_id/version) or use conditional mutations if needed.

Step 8. Consider hybrid pattern for repeat runs - for frequent retraining, export a nightly Parquet snapshot of features to GCS and use Spark over files for faster, reproducible training; keep Bigtable as the serving store and update only the compact serving rows after model compute.

Step 9. Monitor and tune end-to-end - instrument read throughput, per-tablet latency, Spark task skew, connector RPC metrics, and iterate on partitioning, scan caching, and Bigtable node sizing to eliminate hotspots and minimize training time.

Step 10. Safety, reproducibility, and governance - record manifests (which Bigtable ranges were read), seed and schema versions for feature transforms, store models and evaluation metrics in GCS, and provide a tested replay procedure to reproduce training runs exactly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 14. You need to transform raw CSVs from GCS, enrich them, and load them into Bigtable. How would you design this ETL pipeline using Dataproc?

Step 1. Define schema, row-key strategy, and enrichment plan - choose final Bigtable row key (e.g., entityID|timebucket|reversed_ts or salted key), list required columns, and enumerate enrichment sources (lookup tables, external APIs, or join datasets in GCS) so code can compute keys early and avoid hotspots.

Step 2. Provision a Dataproc cluster with Bigtable connectors and autoscaling - create a Dataproc cluster sized for the job, install the Cloud Bigtable HBase client and any Spark libraries, attach a service account with Bigtable and GCS access, and enable autoscaling or create transient cluster to minimize cost.

Step 3. Partition and stage CSVs for parallel reads - split large CSVs into many files or use GCS file wildcard patterns so Spark read parallelism maps to many input partitions; if files are highly skewed, pre-shard them (e.g., split big files) so tasks are balanced.

Step 4. Implement robust Spark ETL job - read CSVs with schema enforcement, validate/clean rows, apply enrichment joins (broadcast small lookup tables, or sharded joins for big datasets), compute final Bigtable row key and only the columns you need, and coalesce/aggregate per-row mutations to minimize write volume.

Step 5. Batch and write efficiently to Bigtable - use the HBase client BufferedMutator or BigtableIO to write mutations in batches (micro-batching), tune mutation batch sizes and max in-flight RPCs, and write with deterministic partitioning so each worker writes to multiple tablets instead of a single hot tablet.

Step 6. Pre-split table and provision capacity before load - sample intended row keys and create split keys to distribute initial tablet ranges; increase Bigtable nodes temporarily to handle ingestion TPS and avoid write throttling.

Step 7. Make the pipeline idempotent and provide retries - design each file/partition write to be retry-safe by using per-record unique keys or version/timestamp checks; record processed file manifests in GCS or a metadata table to avoid double-processing on job restarts.

Step 8. Handle failures with DLQ and reprocessing - on row-level or transform failures, write offending records to a DLQ in GCS with error context; build a small reprocess flow to fix and reapply those records without re-running the whole job.

Step 9. Validate and reconcile after load - run row counts, checksums, and sample-based value checks between source CSVs and Bigtable rows; use a Dataflow/Dataproc job to compare and surface mismatches, and repair via targeted replays.

Step 10. Automate, schedule, and monitor - wrap the job in a workflow (Cloud Composer / Airflow or a scheduled Dataproc job), emit metrics (records/sec, write latency, DLQ size), and clean up the transient cluster after completion; maintain versioned ETL code and run manifests for reproducibility.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 15. A Spark job querying Bigtable is running very slowly. What steps would you take to improve performance?

Step 1. Profile the job and identify hotspots - collect Spark job UI metrics (task durations, shuffle sizes), connector/HBase RPC metrics, per-tablet Bigtable CPU/latency, and check for skewed partitions or stragglers to know if the bottleneck is network, Bigtable throughput, or Spark parallelism.

Step 2. Reduce unnecessary data transfer and RPCs - project only required columns (avoid full-row reads), push down filters to the Bigtable scan when possible, and avoid full table scans by using row-range scans derived from keys or time buckets.

Step 3. Improve parallelism and input splitting - split reads into many balanced partitions by defining explicit key ranges (time buckets or hashed prefixes) so Spark tasks read independent tablet ranges; increase the number of partitions/executors to match Bigtable's ability to serve concurrent reads.

Step 4. Tune connector and scanner settings - increase scanner.caching and scanner.batch to fetch more cells per RPC, adjust hbase.client.scanner.timeout and client threads, and enable asynchronous client APIs so tasks don't block on tiny RPCs.

Step 5. Pre-export heavy reads to columnar files if justified - for large-scale analytics, export Bigtable to Parquet/Avro on GCS (via Dataflow or a bulk export job) and run Spark over those files which gives better IO parallelism and lower latency than many small RPCs.

Step 6. Address key hotspotting and tablet contention - analyze key distribution; if many reads target the same tablet, change row-key design (salt/time-bucket) or pre-split the table so load is spread; consider caching hot ranges in-memory or in Redis if reads are extremely repetitive.

Step 7. Tune Spark resources and data locality - right-size executors (memory/cores), increase parallelism for IO-bound workloads, and avoid big GC pauses; persist intermediate datasets with appropriate storage level (MEMORY_AND_DISK) to prevent recompute overhead.

Step 8. Minimize shuffles and expensive transforms - push down transformations to the read stage where possible, avoid wide dependencies that induce large shuffles, and use map-side aggregations to reduce data movement.

Step 9. Monitor and scale Bigtable capacity - check per-node CPU and latency; increase Bigtable nodes or enable autoscaling to raise read throughput if Bigtable is the limiting resource, and re-run tests after scaling to validate improvement.

Step 10. Iterate with targeted load tests and benchmarking - run controlled experiments changing one variable at a time (scanner.caching, partition count, batch sizes, Bigtable nodes), measure effect on task latency, and converge on the combination that yields acceptable job runtime while keeping cost and latency balanced.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 16. You need to sync operational data from Bigtable into BigQuery for analytics. How would you design the pipeline?

Step 1. Choose sync semantics and cadence - decide whether you need near-real-time (seconds/minutes), micro-batch (e.g., every 1–15 minutes), or daily full loads, and whether you need exactly-once, at-least-once, or eventual consistency for analytics; this guides technology and dedupe strategy.

Step 2. Export a canonical change feed from Bigtable or upstream - prefer emitting change events at the source into Pub/Sub (CDC or application events) that include operation type, primary key, timestamp, and changed fields; if that's impossible, schedule periodic diff/exports (export full rows or key ranges) from Bigtable to GCS.

Step 3. Use Dataflow (Beam) as the streaming/batch transformer - build a Dataflow pipeline that consumes Pub/Sub (or reads GCS exports), normalizes and enriches events, applies schema mapping, performs deduplication/windowing, and writes out partitioned Parquet/AVRO to GCS or streams directly into BigQuery using the Storage Write API.

Step 4. Decide direct streaming vs staged files - for continuous low-latency analytics, write from Dataflow directly into BigQuery Streaming Inserts or Storage Write API with proper batching and idempotency keys; for higher throughput and simpler schema control, write partitioned files to GCS and run load jobs (BigQuery load) for transactional consistency and lower cost.

Step 5. Implement idempotency and dedupe - include a stable event_id and last_updated_ts; use dedupe in Dataflow (stateful windowing) or rely on BigQuery's insertId / dedupe strategies, and for file loads ensure load jobs are idempotent by naming files with run IDs and tracking manifests.

Step 6. Map and evolve schema safely - define a BigQuery target schema and transformation rules (field types, nested structs); handle missing fields, type coercion, and schema evolution by using nullable fields, adding new columns with default nulls, and maintaining a schema registry/version for the pipeline.

Step 7. Partition and cluster BigQuery tables - load into partitioned tables (ingestion_date or event_ts) and add clustering on common query columns (customer_id, device_id) to speed analytics and reduce cost; choose partition granularity (DAY/HOUR) based on query patterns.

Step 8. Provide monitoring, retries, and DLQ - emit metrics for pipeline lag, failed records, BigQuery load errors; route poison messages to a DLQ (Pub/Sub or GCS), surface alerts for backlogs, and provide automated retries or manual reprocessing workflows.

Step 9. Validate data and build reconciliation - implement row counts, checksums, and sampled value comparisons between Bigtable and BigQuery after loads; store manifests and offsets so you can replay ranges and repair mismatches deterministically.

Step 10. Automate and secure the workflow - schedule/trigger pipelines (Cloud Scheduler / Cloud Composer) for batch jobs, grant least-privilege IAM (Dataflow service account Bigtable/BigQuery/GCS permissions), and version pipeline config so the sync is repeatable and auditable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 17. Your hybrid architecture requires Pub/Sub → Bigtable for real-time storage and BigQuery for reporting. How would you orchestrate it?

Step 1. Define a single canonical event stream and schema - have producers publish a canonical event (protobuf/Avro) to Pub/Sub that contains keys, timestamps, event_id, and payload so downstream consumers share the same source of truth and schema evolution is manageable.

Step 2. Split responsibilities with dedicated consumers - implement one low-latency consumer (Dataflow or autoscaled microservices) that writes to Bigtable for serving/operational needs and another (could be same Dataflow job with branching) that prepares/loads data into BigQuery for analytics; keep the transformations for serving vs analytics decoupled.

Step 3. Use a unified Dataflow pipeline with branching for efficiency - create a single streaming Dataflow job that reads Pub/Sub, validates/enriches once, then branches: one branch writes micro-batches to Bigtable using BigtableIO (tuned for low latency), the other writes to BigQuery (via Storage Write API or writes partitioned files to GCS). This reduces duplicate parsing and enrichment cost.

Step 4. Implement event ordering, idempotency, and dedupe - include event_id and version; for Bigtable use compare-and-mutate or last_updated cell checks to avoid duplicate application; for BigQuery use insertId or idempotent file naming so replays/inserts don't create duplicates.

Step 5. Tune latency vs throughput per sink - for Bigtable favor small batches, high concurrency, and anti-hotspotting keys (sharding/time buckets); for BigQuery favor larger micro-batches or file loads to optimize cost and throughput while meeting reporting freshness SLAs (seconds vs minutes).

Step 6. Provide a DLQ and retries for failed records - ensure both branches handle failures: transient errors retried, permanent schema/validation errors sent to a DLQ with context so analysts and engineers can remediate without losing pipeline progress.

Step 7. Ensure schema mapping and transformations are versioned - centralize transformation logic (shared library or service) so both sinks see consistent enrichment; maintain a schema registry and migration plan so updates don't break live serving or reporting.

Step 8. Orchestrate with monitoring and autoscaling - expose end-to-end metrics (Pub/Sub lag, Dataflow processing time, Bigtable write latency, BigQuery load latency), set autoscale policies for Dataflow and Bigtable capacity, and alert on SLO breaches so you can react fast.

Step 9. Support replay and backfill workflows - persist raw events (optionally to GCS) or rely on Pub/Sub retention/backup so you can replay historic ranges to rehydrate Bigtable or backfill BigQuery; design replay idempotency so replays are safe.

Step 10. Document ownership and SLAs - define which team owns the canonical stream, Bigtable serving contracts (P99 latency, TTL), and BigQuery reporting freshness and data quality SLAs; maintain runbooks for incidents and periodic reconciliation jobs to keep both sides consistent.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question: 18. Schema mapping between Bigtable rows and BigQuery tables is causing ingestion issues. How would you resolve it?

Step 1. Formalize a canonical schema contract and mapping spec - document the Bigtable row model (column families, column qualifiers, datatypes, and nullable behavior) and the target BigQuery schema with an explicit one-to-one mapping, type coercion rules, and handling of missing/extra fields.

Step 2. Centralize transformation logic and version it - implement a single, versioned transformation module (in Dataflow or a shared library) that converts Bigtable row representations into BigQuery rows, including type conversions, nested structs, and defaulting rules; deploy new schema versions behind feature flags or controlled rollout.

Step 3. Use an intermediate canonical format (Avro/Proto) - convert Bigtable rows into a strongly-typed Avro or Protobuf message in the pipeline; validating against an Avro/Proto schema makes type errors explicit and easier to manage before loading to BigQuery.

Step 4. Coerce and validate types defensively - implement strict validation rules with clear fallback strategies: if a field can't be parsed, either set it to NULL and tag the row with a validation flag, route it to DLQ, or apply a well-documented coercion (e.g., parse numeric strings, truncate overly long strings) depending on business rules.

Step 5. Handle schema evolution with backward compatibility - when adding/removing fields, ensure BigQuery schema changes are additive (new nullable columns), maintain transform code that can emit older and newer schema variants, and run staging loads to verify downstream queries before switching production.

Step 6. Provide automated schema checks and CI gates - include unit/integration tests that validate sample Bigtable rows map to expected BigQuery rows, run schema compatibility checks during CI/CD, and block pipeline deployment if breaking changes are detected.

Step 7. Emit and monitor detailed error telemetry - surface per-record mapping errors, type mismatch counts, and DLQ sizes with contextual data so you can prioritize fixes and understand impact; keep examples of failing rows for debugging.

Step 8. Support graceful backfills and migrations - when a schema change requires reprocessing, use replayable manifests and idempotent transforms so you can backfill BigQuery from Bigtable (or from exported canonical files) without risking duplicates or partial updates.

Step 9. Communicate changes to stakeholders and consumers - publish a schema changelog and migration playbook for analysts and downstream jobs, coordinate cutovers, and provide a window for consumers to adapt queries before retiring old fields.

Step 10. Iterate with small experiments and rollback plans - deploy mapping changes to a staging dataset or a shadow BigQuery table, validate analytics and dashboards, and have a clear rollback path (replay raw data into previous mapping) so production ingestion remains stable while you improve mappings.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar